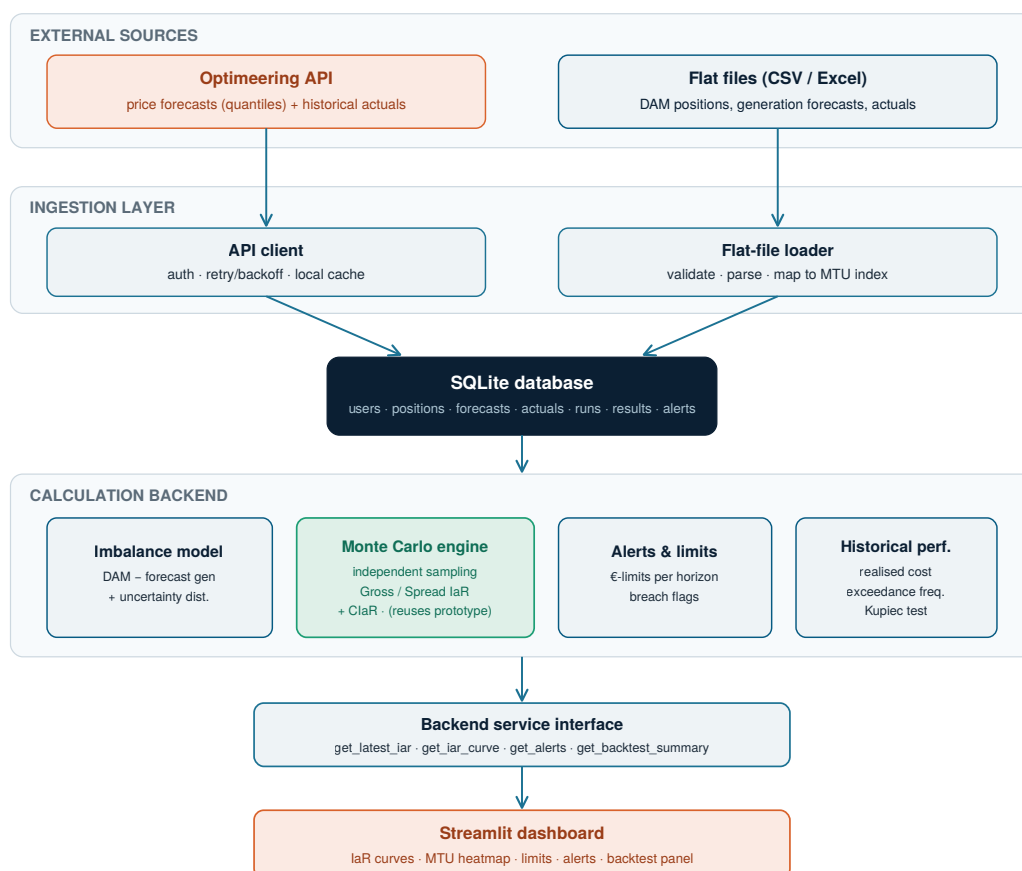


Imbalance at Risk (IaR) — MVP Architecture Design

A pragmatic, local PoC architecture for 2 data scientists. Favours simplicity over production-readiness. Reuses the existing Monte Carlo prototype as the simulation core; everything else is new.

1. Component diagram

Data flows top-to-bottom. External sources are ingested into a single SQLite database, which acts as the integration hub: the calculation backend reads inputs from it and writes results back. A thin service interface is the only thing the Streamlit UI talks to.



The **database as integration hub** is the key structural choice for a PoC: components never call each other directly, they communicate through stored state. This keeps each module independently testable and lets the two developers work in parallel (one on ingestion/DB, one on engine/UI) without tight coupling.

2. Component descriptions

API client (ingestion)

Does: wraps the Optimeering REST API for (a) forward imbalance price predictions with quantile/distribution statistics and (b) historical actual prices, by price area and horizon. Handles API-key auth, retry/backoff, and local response caching.

In: price area, horizon, date range. **Out:** normalised price-forecast quantile series and historical actual-price series,

written to DB.

Library: `requests` + `tenacity` (retry); `requests-cache` or a simple on-disk JSON/parquet cache.

Flat-file loader (ingestion)

Does: reads CSV/Excel for DAM positions, generation forecasts, and actual delivery; validates schema, parses timestamps to an MTU index, writes to DB.

In: file paths. **Out:** validated rows in the positions/forecast/actuals tables.

Library: `pandas` (`read_csv/read_excel`) + `pandera` or a small manual validator for schema checks.

SQLite database

Does: single source of truth for all users, inputs, simulation runs, results, alerts, and backtesting records.

In/Out: read/write from every other component.

Library: `SQLAlchemy` (ORM + schema management) over the stdlib `sqlite3` driver.

Imbalance model (backend)

Does: computes expected portfolio imbalance per MTU (DAM position – forecast generation) and attaches an uncertainty distribution to it — the second input to the simulation.

In: DAM positions + generation forecasts from DB. **Out:** per-MTU imbalance distribution parameters.

Library: `numpy/scipy.stats`.

Monte Carlo engine (backend)

Does: samples independently from the price marginal (Optimeering quantiles) and the imbalance distribution per MTU, builds per-scenario P&L, and reads IaR off the summed-P&L quantile. Produces Gross IaR, Spread IaR, and ClaR/Expected Shortfall by confidence level and horizon. **Reuses the existing prototype's engine** — minus its copula (independence is the MVP assumption).

In: price marginals + imbalance distributions. **Out:** `SimulationRun` + `IaRResult` rows.

Library: `numpy` (vectorised sampling), `scipy.stats` (marginals/quantiles).

Alerts & limits (backend)

Does: evaluates the latest IaR against configurable per-portfolio €-limits (per-MTU, rolling window, remaining-day) and flags breaches with severity.

In: `IaRResult` + limit config from DB. **Out:** Alert rows.

Library: plain Python; config in a DB table (or TOML loaded into it).

Historical performance (backend)

Does: computes realised imbalance cost (actual imbalance × actual price), compares stored historical IaR vintages against realised costs, and tests exceedance frequency (~5% at P95) via a Kupiec POF test.

In: actual delivery + actual prices + stored `IaRResult` vintages. **Out:** `HistoricalPerformanceRecord` rows.

Library: `numpy/scipy.stats` (chi-square for Kupiec), `pandas` for the join.

Backend service interface

Does: a thin Python module exposing the handful of read functions the UI needs, hiding all DB/query detail behind them.

In: UI calls. **Out:** tidy DataFrames/dicts.

Library: plain Python functions over SQLAlchemy queries.

Streamlit dashboard (UI)

Does: renders IaR curves vs limits, a colour-coded MTU/hour risk heatmap, Gross/Spread toggle, confidence + portfolio/area selectors, alert status, and the backtesting calibration panel — per the mockups.

In: service-interface calls only. **Out:** rendered dashboard.
Library: streamlit + plotly (interactive curves/heatmap).

3. Data model

Entities and relationships. A `User` owns one `Portfolio` (1:1 for the MVP); a `Portfolio` is linked to a price area and accumulates time-series inputs, simulation runs, and alerts. Time-series tables are keyed by `(portfolio_id, timestamp)` at MTU resolution.

Entity	Key fields	Relationships
User	user_id (PK), name	owns 1 Portfolio
Portfolio	portfolio_id (PK), user_id (FK), name, price_area (NO1/NO2/SE3)	belongs to User; parent of all series below + runs + alerts
DAMPosition	portfolio_id (FK), timestamp, mwh	per MTU
GenerationForecast	portfolio_id (FK), timestamp, forecast_mwh	per MTU
ActualDelivery	portfolio_id (FK), timestamp, actual_mwh	per MTU (for backtest)
ImbalancePriceForecast	price_area, timestamp, statistic_type (quantile/mean), value	by area; consumed by engine
ActualImbalancePrice	price_area, timestamp, price	by area (for backtest)
SimulationRun	run_id (PK), portfolio_id (FK), run_ts, vintage_ts, n_scenarios, seed, config_json	parent of IaRResults
IaRResult	result_id (PK), run_id (FK), confidence, horizon, iar_type (gross/spread), iar_value, ciar_value	belongs to SimulationRun
Alert	alert_id (PK), portfolio_id (FK), result_id (FK), limit_type, limit_value, breach_ts, severity	raised against an IaRResult vs a limit
HistoricalPerformanceRecord	portfolio_id (FK), period, iar_estimate, realised_cost, exceeded (bool), kupiec_stat	one per backtested period

4. Key design decisions

1. Modelling imbalance uncertainty with no historical error data yet

The brief requires an uncertainty distribution on (DAM – forecast gen) but the team has no historical forecast-error sample at MVP start.

Recommend: a parametric forecast-error model — Gaussian (or scaled-t for heavier tails) centred on the point forecast, with sigma as a *configurable* fraction of forecast or installed capacity. Keep it a single swappable module so the Week-3 backtest can recalibrate sigma against realised exceedance frequency. Simple, explicit, and improvable once error history accrues.

2. Storing and querying simulation results in SQLite efficiently

Many runs × confidence levels × horizons × IaR types could bloat, and naive per-scenario storage would be huge.

Recommend: store *summary* results (IaR/ClaR by confidence + horizon), not raw scenarios. Keep SimulationRun as a parent with a JSON `config` blob and IaRResult as child rows. Index on `(portfolio_id, vintage_ts)` for fast backtest lookups. If a run's full scenario vector is ever needed for a chart, regenerate it from the stored seed rather than persisting 5,000 floats per run.

3. Optimeering API rate limits and caching

The engine and backtester both hit the API repeatedly; redundant calls risk rate limits and slow iteration.

Recommend: wrap the API in one client module with an on-disk cache keyed by (endpoint, area, horizon, date). Forecasts for a given vintage and all historical actuals are immutable once fetched, so cache them permanently; only re-fetch the current live forecast. Add `tenacity` retry with exponential backoff for transience.

4. Structuring the backtesting comparison (estimate vintage vs realised cost)

Backtesting must compare the IaR estimate that was made *at the time* against the cost that actually occurred — vintage alignment is easy to get wrong.

Recommend: stamp every `SimulationRun` with a `vintage_ts` (the as-of time of its inputs). Backtest joins each settled period's realised cost to the IaR estimate whose vintage precedes that period. Store one `HistoricalPerformanceRecord` per period with the estimate, realised cost, and exceedance boolean; the Kupiec test runs over the boolean series.

5. Separating the calculation backend from the Streamlit UI

Streamlit re-runs its whole script on every interaction; embedding simulation there would be slow and tangled.

Recommend: keep all logic in importable backend modules behind the service interface (section 2); Streamlit only calls those functions and renders. No HTTP service needed for a local PoC — direct import is simplest. This separation also makes the backend independently unit-testable and means the engine could later be lifted into a scheduled job or API with no UI rewrite.

6. Reusing the existing prototype — and where to cut it

Recommend: keep the prototype's per-scenario P&L construction, the summed-quantile IaR (not summed per-MTU), and the Gross/Spread + ClaR outputs. **Remove the t-copula** — the MVP assumes price/imbalance independence — and swap mock-data generation for the real marginals (Optimeering quantiles + imbalance model). Be deliberate about fully removing the copula path so no dependence logic stays half-wired.

7. Configuration and secrets

Recommend: Optimeering API key in a `.env` file (via `python-dotenv`), never committed. App/portfolio/limit config in TOML or a config table. Keeps secrets out of the repo with zero ops overhead.

Scope reminder. This architecture deliberately implements the MVP's independence assumption. The background materials' Challenges 1–3 (fat tails beyond supplied quantiles, price ↔ position dependence, cross-MTU autocorrelation) are out of MVP scope. The module boundaries above — especially the swappable imbalance model and the isolated engine — are drawn so dependence can be reintroduced later (e.g. a copula in the engine) without disturbing ingestion, storage, or UI.

5. Module / folder structure

```
iar_mvp/
├── pyproject.toml          # deps, package metadata
├── .env                   # OPTIMEERING_API_KEY (gitignored)
├── config/
│   ├── app.toml           # confidence levels, horizons, scenario count
│   └── limits.toml        # per-portfolio €-limits
├── data/
│   ├── uploads/           # input CSV/Excel (positions, forecasts, actuals)
│   ├── cache/             # cached Optimeering responses
│   └── iar.db             # SQLite database
├── iar/
│   ├── __init__.py
│   ├── db/
│   │   ├── models.py      # SQLAlchemy ORM models (all entities)
│   │   └── session.py     # engine, init_db(), session factory
│   ├── ingestion/
│   │   ├── optimeering_client.py # API client + cache
│   │   └── flatfile_loader.py  # CSV/Excel loaders + validation
│   ├── simulation/
│   │   ├── imbalance_model.py  # DAM – gen + uncertainty dist.
│   │   ├── price_sampler.py    # inverse-CDF sampling of quantiles
│   │   └── engine.py           # Monte Carlo core (from prototype)
│   ├── risk/
│   │   ├── alerts.py          # limits + breach evaluation
│   │   └── backtest.py        # realised cost, exceedance, Kupiec
│   └── service.py             # backend interface the UI calls
├── app/
│   └── dashboard.py           # Streamlit entrypoint (UI only)
├── scripts/
│   └── run_pipeline.py        # CLI: ingest → simulate → store (Week-1 smoke path)
├── tests/
│   ├── test_engine.py
│   ├── test_ingestion.py
│   └── test_backtest.py
└── docs/
    ├── README.md
    ├── data_contract.md      # Optimeering response shape + field map
    └── assumptions.md         # independence, uncertainty model, limitations
```

The split mirrors the diagram: `ingestion/`, `db/`, `simulation/`, `risk/`, and the `app/` UI never reach past `service.py`. `scripts/run_pipeline.py` is the headless end-to-end path that proves the chain before the UI exists.